IBEX RISC-V · 68 SECURITY ASSERTIONS · 9 MODULES · TARGET IEEE ETS 2027

Assertion Quality Ranker

A beginner's guide to how we measure whether a hardware security check is actually good. It covers the idea, the three metrics with real Ibex examples, what other researchers have done, what our red-team and blue-team reviews found, and the experiments still to run.

Advisor approved: SRTLC · AER · SAPC

Comparison only: CWE/CVSS · Trojan category

Review: SAPC and AER definitions need fixing

The idea Key words 1 · SRTLC 2 · AER 3 · SAPC The score New metric ideas What others did Red and blue team Is it novel? Experiments Experiment steps Decisions

Play animations

watched / caught · not watched / missed · test signal · security rule / score

---

START HERE

The idea in five steps

Step 1

A processor

Ibex is a small open-source RISC-V processor. Its design is written as code (RTL).

Step 2

Security checks

We already have 68 *assertions*: small rules such as "user programs may not change machine registers". All 68 are proven true on Ibex (our BEC 2026 paper).

Step 3

Are the checks good?

A check can be true and still weak: it may watch only one way of breaking the rule. Nobody has a formal way to say how strong each check is.

Step 4

Give each a score

Three metrics, computed by the JasperGold tool on the clean design, give every assertion a quality score called **AQS**.

Step 5

Test the score

We hide 39 hardware Trojans in Ibex. If high-scoring assertions catch more Trojans, the score is meaningful.

> **Why it matters:** a chip designer can then say "this set of checks covers our security logic well", or see exactly which checks are weak and what to add.

VOCABULARY

Key words in plain language

RTL

The code that describes a chip's circuits, written in SystemVerilog.

Assertion

A rule written in code that must always hold, e.g. "in user mode, never write `mstatus`". If it is broken, the assertion "fires".

JasperGold

A Cadence tool that checks rules mathematically, for every possible input, instead of running test cases.

FPV

Formal Property Verification. The JasperGold mode that asks "can any input sequence break this rule?"

SPV

Security Path Verification. The JasperGold mode that asks "can data travel from here to there through the circuit?"

Proven / counterexample

Proven means the rule can never break. A counterexample (CEX) is a concrete step-by-step case where it does.

Cone of influence (COI)

All the circuit logic that can affect an assertion. It is the part of the design the assertion "sees".

Hardware Trojan

A small malicious change hidden in a chip design, e.g. a backdoor that grants machine privileges.

Non-vacuous

The assertion's "if" part can actually happen, so it is not passing only because it never gets tested.

`mstatus`

An Ibex machine-mode control register. Changing it from user mode would be a privilege attack.

Metric 1 · Security RTL Coverage (SRTLC)

`JasperGold COI report · 0 new runs`

How much of the security logic do our checks look at?

Everyday picture

A bank has 40 important doors. The security cameras (assertions) can see 30 of them. Coverage is 30 / 40 = 75%. The other 10 doors are unwatched.

1. Which code lines are "security logic"? Those that drive signals named in the expert NS31A security properties. We don't guess them.
2. JasperGold reports each assertion's cone of influence: the lines it can see.
3. Count the security lines that at least one assertion sees.

$$\text{SRTLC} = \frac{\text{security lines seen by some assertion}}{\text{all security lines in the module}} \times 100\%$$



75% 30 of 40 security lines seen (example, CSR module)

Each box is one line of security code. Red boxes are lines no assertion can see, so a Trojan placed there could go unnoticed.

> **Review note:** COI coverage can overestimate what an assertion really checks. Both review teams suggest also reporting *proof-core* coverage, a stricter JasperGold option.

Metric 2 · Assertion Evasion Resistance (AER)

`JasperGold FPV + test signal · 10 runs`

Can someone break the security rule without this check noticing?

Everyday picture: testing a smoke alarm

The goal is that the kitchen never burns unnoticed. The smoke alarm is our check. To test it properly, we let a helper start a small fire anywhere, at any time, in any way. If the kitchen can burn while the alarm stays quiet, the alarm is weak. If every fire, in every place, sets it off, the alarm is strong.

In AER, JasperGold plays the helper, and it tries **every** possibility, not just a few.

The three players (real Ibex example)

`THE SECURITY RULE (P)`

What must never happen. This is the goal, written once.

```
In user mode, mstatus
must never change.
```

`THE ASSERTION UNDER TEST (A)`

One of our 68 checks. AER scores this one.

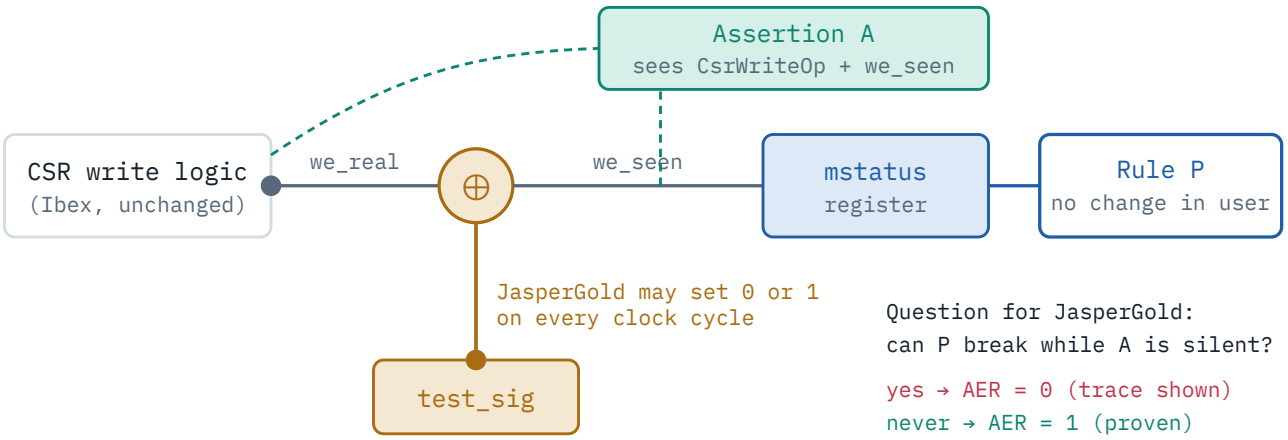
```
UserMode && CsrWriteOp
  |-> !WriteEn_mstatus
```

"If user mode AND a CSR write instruction, then the mstatus write-enable is 0."

THE TEST SIGNAL

An extra input JasperGold controls freely. It can flip the `mstatus` write-enable wire on any clock cycle.

```
we_seen = we_real XOR test_sig
```



The test signal sits on the write-enable wire. The Ibex code itself is not changed; the XOR lives in a small wrapper around it. Dashed green lines show what assertion A looks at.

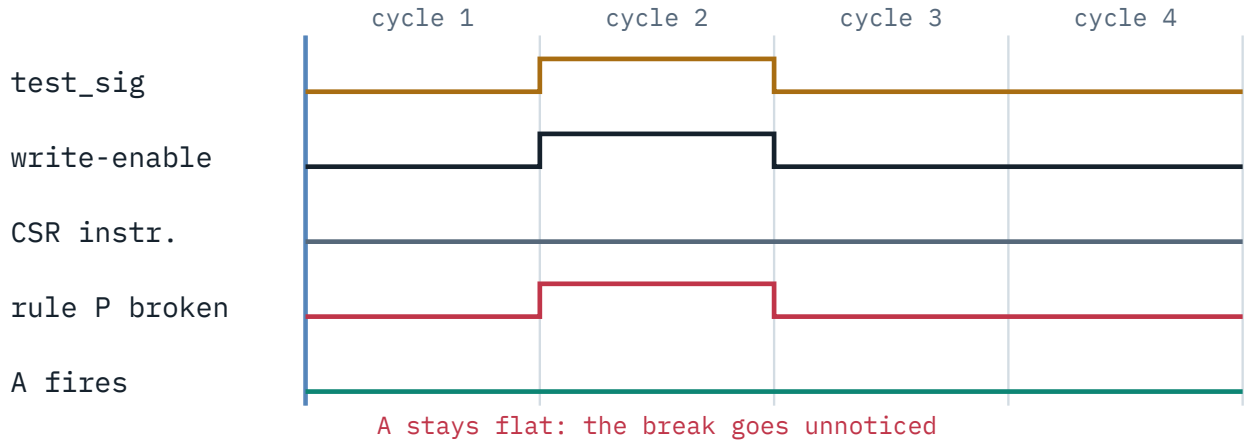
JasperGold then asks one question: **is there any way to set the test signal so that rule P breaks while assertion A stays silent?**

Weak assertion (A) | Stronger assertion

◀ Previous cycle | Next cycle ▶

Press **Next cycle** to walk through the trace one clock cycle at a time.

Clock cycle	Mode	CSR write instruction?	Test signal flips write-enable?	mstatus changed?	Rule P	Assertion A
1	user	no	no	no	OK	silent
2	user	no	**yes**	**yes**	BROKEN	silent
3	user	no	no	no	OK	silent



Result: counterexample, AER = 0

In cycle 2 the write-enable is flipped directly, without any CSR instruction. The "if" part of A (`CsrWriteOp`) is false, so A never checks anything, and mstatus changes in user mode unnoticed. JasperGold shows this exact trace, which tells us precisely why A is weak: **it watches the instruction, not the register.**

```
AER = 1  if no test-signal pattern breaks rule P while A stays silent
AER = 0  if JasperGold finds one (and shows the exact trace)
```

> **Important:** rule P and assertion A must be different. If we used A as its own rule, A could never "break silently" and every assertion would score 1. The review teams suggest taking P from the original NS31A expert property. This is a decision for us (see decisions). The table above is an illustration; real AER values come from the JasperGold runs, which are not done yet.

Metric 3 · Security Assertion Path Coverage (SAPC)

`JasperGold SPV · 10 runs`

Are there roads into the security register that no alarm is watching?

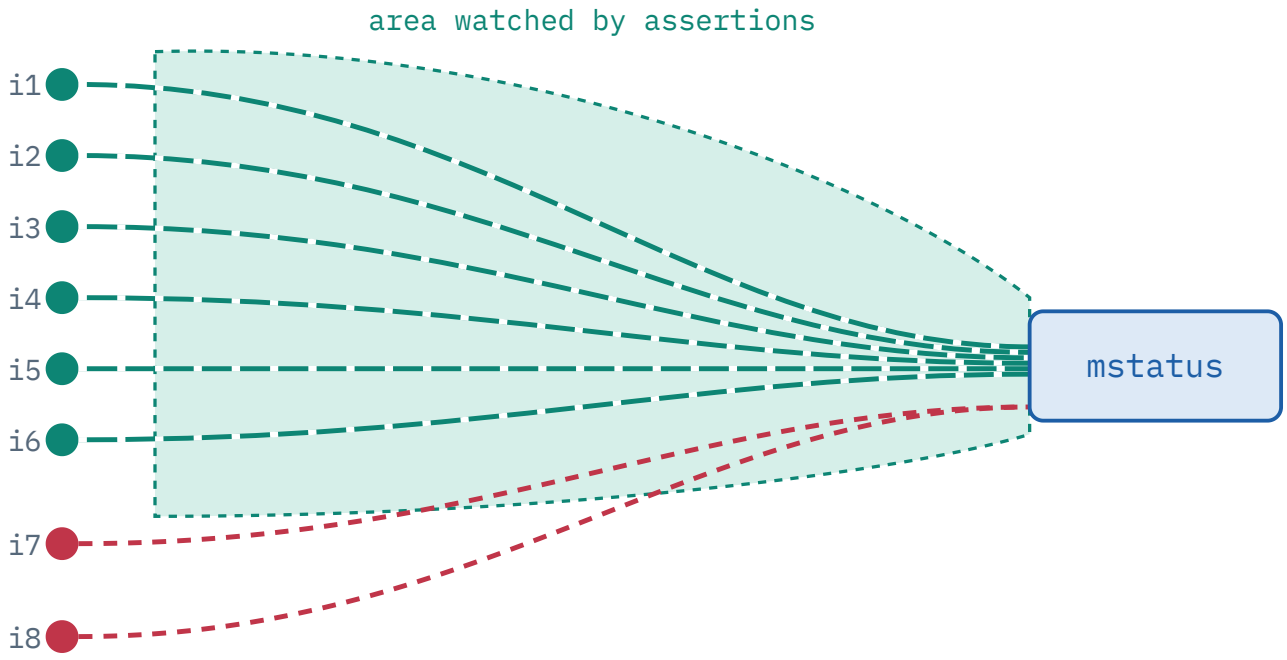
Everyday picture

Eight roads lead to the vault (`mstatus`). For each road we ask whether a car on it would trigger any alarm. Six roads would, two would not. SAPC = 6 / 8 = 75%.

1. JasperGold SPV lists every path from a chip input to a security register.
2. For each path, check whether a change travelling along it reaches some assertion.
3. Paths that reach no assertion are blind spots.

$$\text{SAPC} = \frac{\text{paths into security registers watched by an assertion}}{\text{all paths into security registers}} \times 100\%$$

> **How it differs from AER:** AER only flips the one wire the test signal is attached to. SAPC looks at *all* roads, including ones AER never touched.



75% 6 of 8 roads watched (example)

Dots on the left are chip inputs. Green roads are watched; the two dashed red roads reach `mstatus` with no alarm.

Problem the review found, and the fix

If "watched" just means "the road lies inside the assertion's cone of influence", every assertion that looks at `mstatus` automatically watches *all* roads into it. The number would be almost always 100% or 0% and would just repeat SRTLC.

Fix: for each road, ask SPV whether a change sent down it actually travels through `mstatus` all the way to the assertion's failure output. This follows the method of our co-author's earlier work (Eslami, Ghasempouri, Pagliarini, 2022).

PUTTING IT TOGETHER

The Assertion Quality Score (AQS)

Each metric gives a number between 0 and 1 for every assertion. AQS is their average:

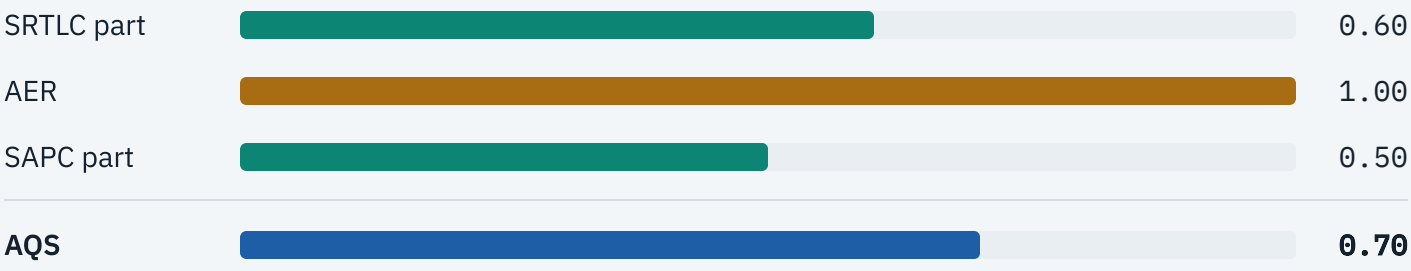
```
AQS = (SRTLC part + AER + SAPC part) / 3
```

How we check the score works: after scoring, we run the same 68 assertions against 39 Trojan-infected versions of Ibex and count how many Trojans each one catches. If assertions with higher AQS catch more Trojans, the score predicts real protection. Statistically this is a *correlation*; the reviews recommend Spearman's rank correlation with confidence intervals.

Beside the score: two descriptive tables

Table A: which known weakness each assertion protects against (CWE number and its CVSS severity, following NIST CSWP 45).

Table B: which Trojan types each assertion targets (Availability, Covert, Denial of Service, Integrity, Leak, Privilege). These describe the assertions; they are not part of the score.



> Example numbers for one assertion, not measured results.

FROM THE RESEARCH · CANDIDATES, NOT YET APPROVED

Suggested new metrics

A research agent searched 21 topics for metrics that could strengthen the paper. It proposed six. Two are worth considering; four are not, because other papers already cover them or they

repeat what we have.

C1 · Evasion Budget (EB): AER as a score, not just 0 or 1

recommended

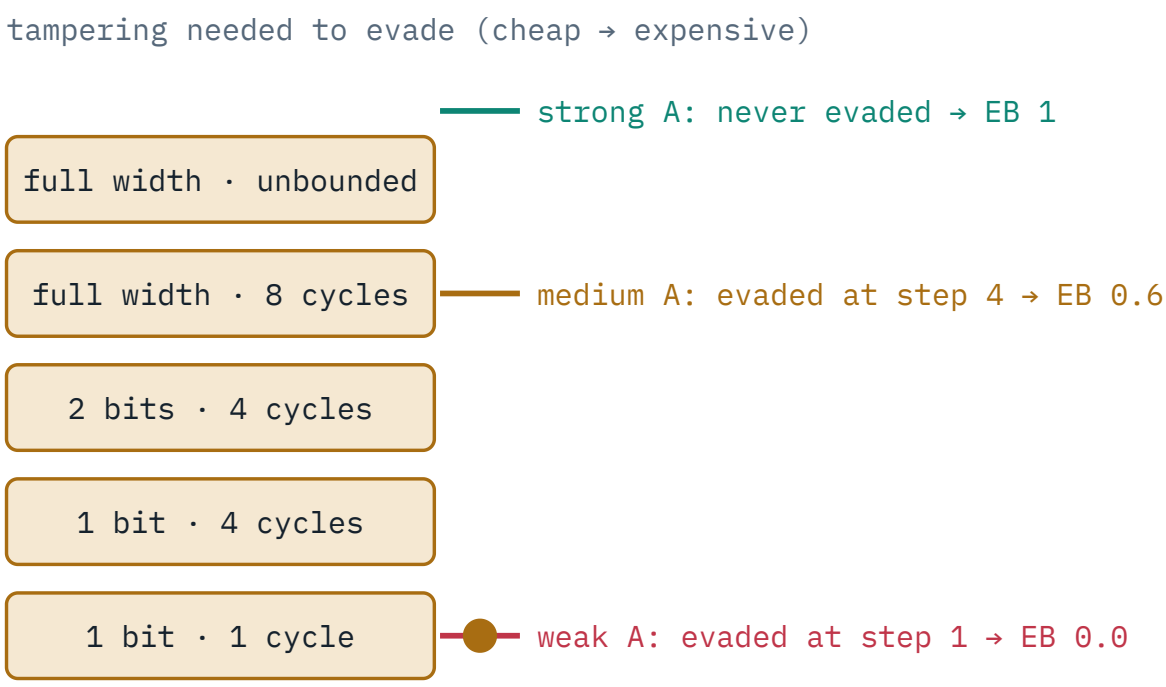
The problem it fixes: with plain AER, almost every assertion gets the same answer (all 0 or all 1). Then the scores tie, and the correlation with Trojan detection has nothing to work with.

The idea: instead of "can the test signal break the rule at all?", ask "how much tampering is needed?". JasperGold is limited step by step: first 1 bit for 1 cycle, then more bits, then more cycles. The assertion's score depends on the smallest tampering that slips past it.

Everyday picture: a lock that opens with a paperclip is worse than one that needs a drill, even though both can be opened.

```
EB = 1                      if no tampering level evades A
EB = (level − 1) / levels   otherwise (cheapest evading level)
```

> **Cost:** no extra JasperGold runs; extra cover checks inside the AER runs. **Novelty:** likely new as a per-assertion score; the "k faults" idea comes from fault-attack research (cite IACR ePrint 2024/247). **Fits the advisor's decision:** it is a better AER, not a 4th metric.



Each step is a bigger tampering budget. Weak assertions break at the bottom step; strong ones survive every step.

C3 · Privilege/Reset Context Coverage (PRCC)

`possible 4th metric, pilot first`

The idea: a RISC-V core runs in different situations: user mode, machine mode, debug mode, and just after reset. An assertion might only ever check anything in machine mode, leaving user-mode attacks unwatched. PRCC counts in how many of these situations the assertion is actually active.

```
PRCC = situations where A is active / situations that can occur
```

> **Cost:** about 10 cheap JasperGold runs. **Novelty:** likely new for formal checks (only simulation tools measure privilege coverage). **Risk:** if most assertions are active everywhere, every score is 1 and the metric is useless, so first try it on two modules (CSR and debug). Links to CWE-1271 (insecure reset values).

Example assertion	user	machine	debug
just after reset	not active	active	not active
normal running	active	active	not active

3 / 6 PRCC = 0.5 (illustration)

ID	Candidate	In plain words	Extra runs	Novelty	Verdict
C1	Evasion Budget	How much tampering is needed to slip past the assertion	0	likely new	`adopt as graded AER`
C2	Detection Latency	How many clock cycles until the	0	likely new	`optional; most Ibex checks fire in 0–1 cyc`

ID	Candidate	In plain words	Extra runs	Novelty	Verdict
		assertion fires after the rule breaks			
C3	Privilege/Reset Context Coverage	In which modes and reset phases the assertion is active	≈10	likely new (formal)	`pilot on CSR + debug`
C4	Rarity-weighted coverage	Count rarely used security code more heavily	≈10	partly done before	`reject: only reweights SRTLC`
C5	Unique contribution	What is lost if this one assertion is removed	0	low	`tie-breaker only`
C6	Information-flow coverage	Can secret data reach an output past every assertion	≈12	done before (Meza & Kastner; Eslami 2022)	`reject: overlaps SAPC`

> **Recommendation:** keep 3 metrics, make AER graded with the Evasion Budget, and add PRCC only if the two-module pilot shows real differences between assertions. Full report:

> docs/research/new_metric_ideas.md.

LITERATURE · 72 PAPERS SURVEYED

What other researchers have already done

Our literature survey found work close to every one of our metrics. That is normal. It tells us which ideas to credit and where our contribution really lies. The papers below are the most important, grouped by the question they answer.

Closest to AER

"Does the check notice when the design is changed?"

Chockler, Kupferman, Vardi et al., 2001

Change the design slightly (mutation) and see whether the property notices. The classic idea behind AER.

Kupferman, CONCUR 2006

"Sanity checks" after a proof: vacuity and coverage. The framing for our whole paper.

Krishnamurthy et al. 2026 (arXiv 2606.21451); HierSVA 2026 (arXiv 2606.13706)

Score LLM-written assertions by how many injected faults they catch.

For us: AER differs because it checks *all* flip patterns in one proof, against a security rule.

Chuah, Appold, Leinmüller: MBMV 2024; RISC-V Summit Europe 2025; ISQED 2026 (ISQED page)

Formal Trojan detection on a RISC-V core with Jasper. The red team found the same "XOR free input" trick in their 2025 abstract.

For us: cite them as the origin of the mechanism; our addition is a score per assertion.

Closest to SAPC

"Can a change travel to the check?"

Eslami, Ghasempouri, Pagliarini, ISQED 2022 (arXiv 2201.01130)

Uses JasperGold SPV to count how many circuit points can reach each assertion ("Security Coverage") and ranks assertions by it. Scores were about 5–7% because they divide by the whole circuit.

For us: our own group's work. SAPC narrows it to security registers and tests it against Trojans.

Eslami et al., SCARF, IEEE Trans. Computers 2024 (arXiv 2402.12162)

Journal version of the same per-assertion security coverage.

Meza & Kastner, 2023 (arXiv 2304.08263)

Coverage metrics based on how information flows through hardware.

Closest to SRTLC

"How much of the design does the check see?"

JasperGold coverage app (Cadence)

Built-in cone-of-influence and proof-core coverage. Cadence's own material says COI overestimates.

For us: SRTLC's new part is counting only expert-flagged security lines. Also report proof-core.

LAsset (DATE 2026); SAIF (VTS 2021)

Find which signals are security "assets". This decides what counts as security logic.

Closest to AQS ranking and our validation

"Which checks are best, and do they catch Trojans?"

Ayalasomayajula et al., HOST 2024; Assert-O 2025; Hertz et al., ASP-DAC 2019

Earlier ways to rank assertions (by severity, by combined merit scores).

For us: AQS ranks by formal evidence, not severity labels.

Imtiaz, Reinsalu, Ghasempouri, ISCAS 2025 (arXiv 2502.10194)

Our own earlier paper: Ibex assertions tested on Trojans with the TDER and TPI measures.

For us: reuse and cite TDER/TPI; don't present them as new.

Inozemtseva & Holmes, ICSE 2014

In software testing, coverage correlates only weakly with bug-finding once test size is controlled.

For us: a warning. Control for assertion size when correlating AQS with detection.

Trojan benchmarks

"Where do test Trojans come from?"

TrustHub (Shakya et al., 2017)

The standard public Trojan benchmark and its categories.

Krieg, ICCAD 2023

Found only 3 of 83 TrustHub designs are real Trojans.

For us: say "TrustHub categories, new Ibex code", not "TrustHub Trojans".

Damljanovic et al., DDECS 2021; RISC-V-Dilemma-1, 2025

Earlier processor Trojan benchmarks.

For us: don't claim the first RISC-V Trojan benchmark.

> Full list with links: `docs/research/literature_survey.md`. About 15 entries are marked unverified and must be checked before citing.

NOVELTY REVIEW

Red team and blue team

Red team: the hostile reviewer

Searched about 30 sources and tried to destroy every novelty claim, the way a tough ETS reviewer would.

Blue team: the defence lawyer

Searched about 25 sources and built the strongest *honest* case, plus the wording and experiments that survive the red team.

Item	Red team says	Blue team says	In plain words
SRTLC	Partly done before; COI coverage is a standard tool feature.	Defensible: the expert-defined security denominator is new.	`keep` Call it an adaptation. Add proof-core coverage.
AER	Mechanism already published (Chuah 2025); rule P undefined.	A formal relative of mutation coverage, but new as a per-assertion security score.	`fix` Define P separately give a graded score instead of 0/1.
SAPC	Done by our co-author (2022), and may simply repeat SRTLC.	Valid if "watched" means a real path to the assertion's output.	`must fix` Redefine as functional paths; credit Eslami 2022.
AQS ranking	Assertion ranking exists already.	New: combines behaviour (FPV) and paths (SPV) in one score.	`keep` Claim the combination, never "first"

Item	Red team says	Blue team says	In plain words
Score predicts Trojans	Looks new, but small data and our own Trojans.	The strongest claim, if done carefully.	`fix` Rank statistics, baselines, held-out Trojans.

All eight attacks from the red team

#	Attack, in plain words	How serious	What answers it
1	SAPC repeats SRTLC: measured with the cone of influence, it is almost always 100% or 0%.	**fatal if true**	Count real SPV paths to the assertion's output; show SRTLC and SAPC do not correlate strongly.
2	AER's rule P is not defined. If P is the assertion itself, AER is always 1.	**fatal if unclear**	Take P from the NS31A expert property; show one real evasion trace in the paper.
3	AER is Chuah's model plus a standard tool trick, and 0/1 makes most assertions tie.	major	Credit Chuah; use the graded Evasion Budget; show the scores really vary.
4	The Trojans are not independent: we wrote both assertions and Trojans, and our earlier paper caught 100%.	major	Freeze Trojans before scoring; add outside Trojans; show some Trojans ar missed.
5	Weak statistics: 68 assertions, small counts, many ties, module effects.	major	Spearman with confidence intervals; per-module analysis; compare with simple baselines.
6	COI coverage is a known weak proxy, and NS31A experts may have missed signals.	major	Also report proof-core coverage; list the NS31A limit as a limitation.
7	Refinement is tested on the same Trojans that guided it ("train on test").	major	Split the Trojans; test refined assertions only on the held-out part.
8	Equal weights and the CWE/CVSS mapping look arbitrary.	minor now	Ablation and random-weight stability; CWE stays a descriptive table only.

> **Also raised:** report tool run times (ETS reviewers care about test cost); 4 of the 9 modules share one file, so there are really about 6 independent modules; explain why ranking is needed if our earlier paper detected every Trojan.

The four problems to solve first

MUST FIX · SAPC

SAPC may count nothing new

Measured with the cone of influence, any assertion that looks at a register "watches" every road into it.

Fix: count only roads where SPV shows a change really reaches the assertion's failure output.

MAJOR · AER

Which rule is P?

If P is the assertion itself, AER is always 1. A fully free test signal may also make almost every assertion score 0.

Fix: take P from the NS31A expert property; score how many flip points or how much tampering the assertion catches (graded).

MAJOR · VALIDATION

The Trojan test could look rigged

We wrote the Trojans ourselves, the sample is small, and refinement would be tested on the same Trojans that guided it.

Fix: freeze the Trojans first, keep some aside for the final test, compare with simple baselines.

MAJOR · POSITIONING

No "first" claims

Every single metric has a close ancestor in the literature.

Fix: credit Chuah and Eslami openly and claim the combination plus validation.

OUR HONEST CONTRIBUTION

Is this idea novel? An honest answer

Part	Already done by others?	What is ours
SRTLC	Mostly: coverage tools measure cone-of-influence coverage	Counting only expert-defined security code
AER	The mechanism: Chuah et al.; mutation coverage since 2001	A score per assertion; graded by tampering budget (if adopted)
SAPC	Very close: our co-author's 2022 SPV security coverage	Narrowed to security registers and tested against Trojan

Part	Already done by others?	What is ours
Ranking assertions	Yes: HOST 2024, ASP-DAC 2019, Assert-O 2025	Ranking by formal evidence instead of labels
All of it together	**Not found in about 70 papers**	**Behavioural + structural evidence per assertion, computed before any Trojan, then shown to predict detection on a real RISC-V core**

Verdict: moderate novelty. It is enough for a 6-page ETS paper if it is framed as "combination plus validation" and credits Chuah and Eslami openly. It is not a breakthrough, and claiming "new metrics" would be rejected. The real risk is the result: if AQS does not predict Trojan detection better than simple baselines (such as cone size), the main claim falls away. Fixing SAPC and defining rule P must come first.

How to raise novelty from moderate to high

More metrics will not do it. What lifts the paper is turning the metrics into a **guarantee** and a **method**.

Idea	In plain words	Why it raises novelty	Fits the deadline?
1 · Security sufficiency certificate	Prove: if every security signal has full Evasion Budget and full SAPC, then any Trojan that tampers with one security signal within the budget (k bits, d cycles) is *guaranteed* to be caught.	Turns scores into a formal promise: "passing these metrics means the processor is secure against this class of attack". We found nothing like it.	`yes, for a bounded threat model`
2 · Counterexample-guided repair	Each AER counterexample is an exact evasion trace. Build the new assertion from that trace, prove it, repeat until no evasion is left.	Replaces generic templates with a loop driven by security evidence. Existing repair work uses	`light version`

Idea	In plain words	Why it raises novelty	Fits the deadline?
		mutants, not evasion traces.	
3 · Security margin as the headline	Report for each assertion the smallest tampering (bits × cycles) that evades it.	A new, easy-to-grasp quantity, and the graded scores make the correlation meaningful.	`yes, 0 extra runs`
4 · Generalisation	Test on Trojans from other groups or on a second core (e.g. CVA6).	Mostly adds credibility, less novelty.	`future work`

> With ideas 1 and 3 (and 2 if time allows), the contribution becomes: *a formal security margin per assertion, a sufficiency certificate for a stated threat model, and evidence that both predict real Trojan detection.* That is a high-novelty framing that still rests on the same three metrics.

What we can claim

Proposed novelty sentence (blue team): AQS is a per-assertion security quality score computed by formal analysis on Trojan-free RTL, combining coverage of expert-defined security signals (SRTLC), FPV-based evasion resistance (AER) and SPV-based functional-path coverage (SAPC). On 68 proven Ibex assertions across 9 security modules, AQS computed before any Trojan exists correlates with detection on an independent 39-Trojan benchmark, and AQS-guided refinement improves detection on held-out Trojans.

- **Safe to say:** "to our knowledge, the first to combine FPV evasion and SPV path evidence in one per-assertion score".
- **Our cleanest new insight, if the data shows it:** an assertion can be behaviourally strong (AER high) and still leave roads unwatched (SAPC low).
- **Never say:** "first metric", "first RISC-V Trojan benchmark", or that AQS "proves" detection. Say "correlates with".

NEXT STEP

Five stages and the experiments to run

Stage 1 · AVS

Validation

Done in BEC 2026: 68 assertions proven and non-vacuous.

`0 new runs`

Stage 2 · AQRS

Ranking

SRTLC, AER, SAPC on the clean design; AQS and both tables.

`10 FPV + 10 SPV`

Stage 3 · TES

Trojan test

Assertions on 39 Trojan versions; does AQS predict detection?

`39 FPV`

Stage 4 · ARS

Refinement

Add template assertions for the gaps found. Python, no AI.

`0 runs`

Stage 5 · RAVS

Re-test

Re-run on held-out Trojans and measure the gain.

`39 FPV`

ID	Experiment	Stage · tool	Runs	What it produces	Status
E1	SRTLC per module and per assertion (COI, plus proof-core)	AQRS · Python	0	Bar chart per module; list of unwatched security lines	core

ID	Experiment	Stage · tool	Runs	What it produces	Status
E2	AER with test-signal wrappers, one per bind file	AQRS · FPV	10	AER per assertion; the evasion trace for each weak one	core
E3	SAPC functional paths into each security register	AQRS · SPV	10	Path coverage per module; list of unwatched roads	core
E4	AQS ranking of all 68 assertions	AQRS · Python	0	Ranked table, best and weakest per module	core
E5	CWE/CVSS and Trojan-category tables	AQRS · Python	0	Tables A and B	core
E6	Run unchanged assertions on the 39 Trojan designs	TES · FPV	39	Detection heat map; TDER, WTDR, AAD	Trojans
E7	Does AQS predict detection? (Spearman with confidence intervals)	TES · Python	0	Scatter plot; correlation with and without size control	core
E8	Baselines: cone size, number of signals, random ranking	TES · Python	0	Does AQS beat simple predictors?	proposed
E9	Ablation: drop one metric at a time; random weights	TES · Python	0	Is each metric needed? Is the ranking stable?	proposed
E10	Head-to-head: AER vs mutation kill rate; SAPC vs Eslami coverage	AQRS · FPV/SPV	≈10	One worked case the older metric misses	proposed
E11	Refine assertions from the gap report	ARS · Python	0	New template assertions per gap	core
E12	Re-test refined assertions on held-out Trojans	RAVS · FPV	39	Before/after detection on Trojans not used for refinement	Trojans

HOW TO RUN IT

Experiment steps, in order

Each step lists what to do and what comes out. Steps 0 and 1 happen once; everything else runs per module.

Step	What to do	Where	Output
`0 · Freeze`	Commit the 39 Trojans and the 68 assertions with a date tag **before** computing any score, so nobody can say they were tuned to fit. Split the Trojans into a refinement half and a held-out half.	laptop · git	Tagged commit Trojan split list
`1 · Inputs`	Copy the 10 bind files and the JasperGold coverage reports from the BEC 2026 work into `assertions/translated/` and `results/step1/`. List the NS31A security signals per module.	laptop	Inputs ready; security signal list
`2 · SRTLC`	Python reads the coverage reports, marks which security lines each assertion sees (cone of influence and proof core), and computes SRTLC per module and per assertion.	laptop · Python	`srtlc.csv`; bar chart
`3 · AER`	For each bind file: write a wrapper that XORs a test signal into each security signal; write rule P from the NS31A property; run JasperGold FPV asking "P broken while A silent?". Add Evasion Budget cover checks (1 bit/1 cycle → full/unbounded).	server · FPV · 10 runs	`aer.csv`; evasion traces
`4 · SAPC`	For each security signal, ask JasperGold SPV for paths from every input through the signal to each assertion's failure output. Count paths reaching some assertion.	server · SPV · 10 runs	`sapc.csv`; unwatched pat list
`5 · AQS`	Python averages the three scores per assertion and ranks all 68. Build tables A (CWE/CVSS) and B (Trojan categories). Write `gap_report.json`.	laptop · Python	Ranked table; gap report
`6 · Trojans`	Run the unchanged bind files on each of the 39 Trojan designs with JasperGold FPV. A counterexample means the assertion caught the Trojan.	server · FPV · 39 runs	Detection matrix; TDER, WTDR, AAD
`7 · Validate`	Spearman correlation of AQS vs number of Trojans caught, with confidence intervals; repeat with module and cone-size controls; compare with baselines (cone size, signal count, random); drop one metric at a time.	laptop · Python	Scatter plot; correlation table

Step	What to do	Where	Output
8 · Refine	Python reads the gap report and adds template assertions; each new one must be proven and non-vacuous on clean Ibex before use.	laptop + server	Refined bind files
9 · Re-test	Run the refined bind files on the **held-out** Trojans only, plus a control that adds the same number of assertions on random signals.	server · FPV	Before/after detection; control comparison
10 · Report	Record tool run times for every JasperGold run; collect figures for the paper.	laptop	Paper figures and tables

> **Pilot first:** run steps 2–4 on two modules (CSR and debug) before all nine. That shows quickly whether SAPC and AER give scores that actually differ between assertions.

BEFORE EXPERIMENTS START

Decisions for Sharjeel and the advisor

1. **SAPC:** redefine "watched" as a functional SPV path to the assertion's failure output?
2. **AER:** take rule P from the NS31A expert property? Graded score or 0/1?
3. **Statistics:** Spearman with confidence intervals, and keep some Trojans aside for the final re-test?
4. **Citations:** how do the BEC 2026 paper (68 assertions, 9 modules) and the ISCAS 2025 paper (5 modules) relate?

Sharjeel Imtiaz, Uljana Reinsalu, Tara Ghasempouri · Tallinn University of Technology · Estonian Research Council PSG837

Input assertions from *AutoTrans* (BEC 2026). All numbers on this page are illustrative; no Stage 2–5 results exist yet. Full reports: `docs/research/`.